Listing 7.20. `pop()` modified to allow helping on the `push()` side

```
template<typename T>
class lock_free_queue
{
private:
    struct node
    {
        std::atomic<T*> data;
        std::atomic<node_counter> count;
        std::atomic<counted_node_ptr> next;     <-1
    };
public:
    std::unique_ptr<T> pop()
    {
        counted_node_ptr old_head=head.load(std::memory_order_relaxed);
        for(;;)
        {

            increase_external_count(head,old_head);
            node* const ptr=old_head.ptr;
            if(ptr==tail.load().ptr)
            {
                return std::unique_ptr<T>();
            }
            counted_node_ptr next=ptr->next.load();         <-2
            if(head.compare_exchange_strong(old_head,next))
            {
                T* const res=ptr->data.exchange(nullptr);
                free_external_counter(old_head);
                return std::unique_ptr<T>(res);
            }
            ptr->release_ref();
        }
    }
};
```

As I mentioned, the changes here are simple: the `next` pointer is now atomic ❶, so the `load` at ❷ is atomic. In this example, you're using the default `memory_order_seq_cst` ordering, so you could omit the explicit call to `load()` and rely on the load in the implicit conversion to `counted_node_ptr`, but putting in the explicit call reminds you where to add the explicit memory ordering later.

The code for `push()` is more involved and is shown here.